LABORATORIO DE IDS/IPS

Despliegue y configuración de Suricata

Detección y Prevención de Intrusiones en Red

Módulo	Redes III — Unidades 2 y 3
Actividad	Laboratorio práctico IDS/IPS — Suricata en modo inline/activo
Calificación máxima	10 puntos (5 base + 5 extra)
Entrega	Práctica: revisión en clase · Informe: PDF
Nombre del alumno/a	
Fecha	

⚠️ **Aviso Ético y Legal — Lee antes de continuar**

Realizar ataques DoS contra sistemas reales o redes de terceros sin autorización expresa es ILEGAL en la mayoría de países y puede conllevar sanciones penales graves.

Toda la actividad de este laboratorio se realiza EXCLUSIVAMENTE en una red interna aislada (Internal Network de VirtualBox/VMware), sin conexión a Internet ni a redes externas.

Contexto y objetivos

En este laboratorio desplegamos Suricata como IPS en modo inline entre la máquina atacante y la máquina víctima, de manera que todo el tráfico pasa físicamente por el motor de inspección. Escribiréis reglas propias para detectar y bloquear cada uno de los ataques del temario, y verificaremos en tiempo real que Suricata los detiene.

Objetivos

- Instalar y configurar Suricata en modo IPS inline con NFQ (Netfilter Queue).
- Escribir reglas Suricata propias para cada tipo de ataque del temario.
- Verificar experimentalmente la diferencia entre tráfico con y sin IPS activo.

Criterios de evaluación

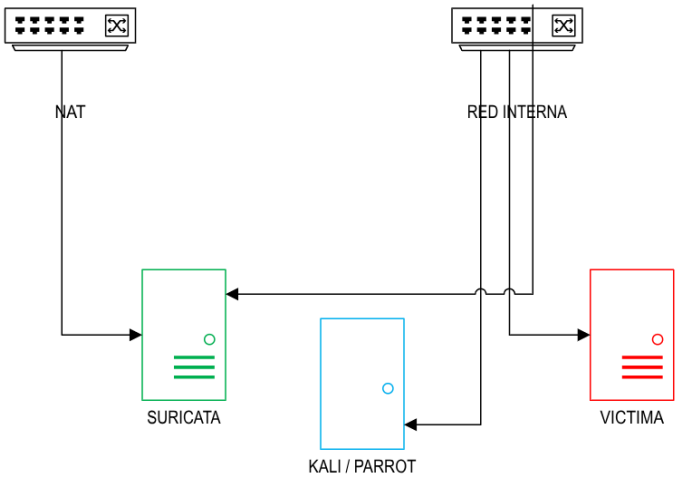
Criterio	Puntos	Descripción
Punto 1: Instalación y configuración base de Suricata	**2 p**	Suricata en modo inline funcional; tráfico legítimo sin interrupciones
Punto 2: Detección y bloqueo DoS/DDoS	**2 p**	Reglas activas para SYN, ICMP y HTTP Flood; capturas htop
Informe PDF	**1 p**	
TOTAL PROPUESTA A	**5 puntos**	

Si quieres obtener los otros 5p tienes que:

Criterio	Puntos	Descripción
Punto 3: Detección ARP Spoofing + debate MITM	**2 p**	Regla ARP personalizada + argumentario escrito sobre limitación en L2
Punto 4: Hardening + auditoría EVE JSON	**3 p**	Threshold configurado, log EVE activo, análisis de un evento por categoría
TOTAL PROPUESTA A	**5 puntos**	

Topología del laboratorio

El laboratorio utiliza tres máquinas virtuales en una red interna aislada. Suricata se instala en una VM intermedia que actúa como puente entre el atacante y la víctima: todo el tráfico atraviesa el motor de inspección antes de llegar al destino.



Máquina	Sistema operativo	IP	Rol en el laboratorio
VM Atacante	Kali Linux	192.168.100.30	Genera ataques: hping3, ab, arpspoof, nmap
VM Suricata	Debian 13	192.168.100.10	Motor IPS + generación de logs EVE JSON
VM Víctima	Debian 13	192.168.10.20	Apache2 + htop + ss (ataques realizados)

Configuración de red en VirtualBox

Red interna [labnet]

- VM Kali (Atacante)
- VM Víctima
- VM Suricata

Red [NAT]

- VM Suricata

Desarrollo del laboratorio

(1) Instalación y configuración básica de Suricata ⏱ 30 min

nstalar Suricata en la VM intermedia

Nota: En este link tienes información oficial:
https://docs.suricata.io/en/suricata-7.0.2/quickstart.html#installation

VM Suricata (192.168.100.1)

```
# Actualizar repositorios e instalar Suricata
sudo apt update && sudo apt install suricata -y

# Verificar la versión instalada
suricata --version
suricata version 7.x.x RELEASE

# Instalar herramientas adicionales
sudo apt install -y iptables htop jq
```

Configurar modo inline con NFQ

El modo NFQ (Netfilter Queue) permite a Suricata recibir paquetes del kernel vía iptables, inspeccionarlos y decidir si los acepta o descarta. Es el modo IPS real: el paquete no llega al destino hasta que Suricata da el visto bueno.

Nota: Tienes información aquí:
https://docs.suricata.io/en/suricata-7.0.2/configuration/suricata-yaml.html#nfq

Configuración NFQ inline — VM Suricata

```
# Editar la configuración principal de Suricata
sudo nano /etc/suricata/suricata.yaml

# Localizar la sección 'nfq' y asegurarse de que está habilitada:
nfq:
  mode: accept   # acepta por defecto; las reglas decidirán el DROP

# Redirigir el tráfico FORWARD a la cola de Suricata con iptables
sudo iptables -I FORWARD -j NFQUEUE --queue-num 0

# Habilitar IP forwarding (para que la VM actúe de puente)
sudo sysctl -w net.ipv4.ip_forward=1
echo 'net.ipv4.ip_forward=1' | sudo tee -a /etc/sysctl.conf
```

Verificar que el tráfico legítimo está OK

Verificación de tráfico legítimo — VM Atacante

```
# Iniciar Suricata en modo inline (desde la VM Suricata)
sudo suricata -c /etc/suricata/suricata.yaml -q 0 --init-errors-fatal &

# Desde la VM Atacante: verificar que la víctima responde
ping -c 4 192.168.10.20
4 packets transmitted, 4 received, 0% packet loss  ← tráfico OK

curl -s -o /dev/null -w 'HTTP %{http_code}\n' http://192.168.10.20/
HTTP 200  ← Apache responde correctamente
```

✅ Criterio de superación del punto (1)

Suricata arranca sin errores, iptables redirige el tráfico a la cola, y el ping y HTTP desde el atacante llegan a la víctima sin pérdida de paquetes.
Captura de pantalla obligatoria para el informe.

(2) DoS/DDoS — Detección y bloqueo

⏱ 30 min

Descargar las reglas Emerging Threats

VM Suricata — descarga de reglas

```
# Actualizar el conjunto de reglas con suricata-update
sudo suricata-update

# Verificar que las reglas se han cargado correctamente
sudo ls /var/lib/suricata/rules/ | head -5
```

Escribir reglas propias para SYN Flood, ICMP Flood y HTTP Flood

Crea el fichero de reglas personalizadas. Cada regla debe incluir un comentario que explique su lógica: protocolo vigilado, umbral elegido y acción tomada.

Reglas personalizadas — lab-custom.rules

```
sudo nano /etc/suricata/rules/lab-custom.rules

# ── Regla 1: SYN Flood ──────────────────────────────────────────────
```

```
# Detecta más de 200 paquetes SYN al puerto 80 en 5 segundos desde la misma
IP
alert tcp any any -> $HOME_NET 80 (msg:"LAB SYN Flood detectado"; flags:S;
threshold:type both,track by_src,count 200,seconds 5; sid:9000001; rev:1;)

# ── Regla 2: ICMP Flood ──────────────────────────────────────────────
# Detecta más de 100 Echo Request ICMP en 2 segundos
alert icmp any any -> $HOME_NET any (msg:"LAB ICMP Flood detectado";
itype:8; threshold:type both,track by_src,count 100,seconds 2; sid:9000002;
rev:1;)

# ── Regla 3: HTTP Flood ──────────────────────────────────────────────
# Detecta más de 500 peticiones GET en 10 segundos desde el mismo origen
alert http any any -> $HOME_NET 80 (msg:"LAB HTTP Flood GET excesivo";
http.method; content:"GET"; threshold:type both,track by_src,count
500,seconds 10; sid:9000003; rev:1;)
```

Recarga de reglas en caliente

```
# Añadir la referencia al fichero de reglas en suricata.yaml
echo '  - /etc/suricata/rules/lab-custom.rules' | sudo tee -a
/etc/suricata/suricata.yaml

# Recargar las reglas sin detener Suricata
sudo kill -USR2 $(pidof suricata)
```

Lanzar los ataques y verificar los bloqueos

Desde la VM Atacante, lanzad cada ataque en secuencia con la VM Víctima mostrando htop. Observad cómo Suricata descarta los paquetes maliciosos y la CPU de la víctima se mantiene estable:

VM Atacante — lanzamiento de ataques

```
# ── Ataque 1: SYN Flood ──────────────────────────────────────────────
sudo hping3 -S --flood -V -p 80 192.168.10.20

# ── Ataque 2: ICMP Flood ─────────────────────────────────────────────
sudo hping3 --icmp --flood 192.168.10.20

# ── Ataque 3: HTTP Flood ─────────────────────────────────────────────
ab -n 10000 -c 500 http://192.168.10.20/
```

En la VM Suricata, verificar las alertas generadas en tiempo real:

VM Suricata — monitoreo en tiempo real

```
# Ver alertas en tiempo real filtrando el log EVE JSON
sudo tail -f /var/log/suricata/eve.json | jq 'select(.event_type=="alert") |
{src_ip,dest_port,firma:.alert.signature}'

# Ejemplo de salida esperada:
{"src_ip":"192.168.10.10","dest_port":80,"firma":"LAB SYN Flood detectado"}
{"src_ip":"192.168.10.10","dest_port":null,"firma":"LAB ICMP Flood
detectado"}
{"src_ip":"192.168.10.10","dest_port":80,"firma":"LAB HTTP Flood GET
excesivo"}
```

(3) Spoofing y MITM — Detección y debate

⏱ 30 min

Ataque ARP Spoofing desde la VM Atacante (Kali)

VM Atacante — ARP Spoofing

```
# Habilitar IP forwarding en Kali (para mantener la conexión activa)
echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward

# Instalar dsniff si no está disponible
sudo apt install -y dsniff

# Terminal 1: engañar a la víctima (decirle que somos el gateway)
sudo arpspoof -i eth0 -t 192.168.10.20 192.168.10.1

# Terminal 2: engañar al gateway (decirle que somos la víctima)
sudo arpspoof -i eth0 -t 192.168.10.1 192.168.10.20

# Verificación en la VM Víctima — tabla ARP modificada:
arp -n
192.168.10.1  ether  xx:xx:xx:xx:xx:xx  ← MAC del atacante!
```

Regla Suricata para detectar ARP Spoofing

Añadir esta regla al fichero lab-custom.rules. Detecta respuestas ARP no solicitadas (gratuitos ARP), que son la firma del envenenamiento de tabla ARP:

Regla ARP — añadir a lab-custom.rules

```
# ── Regla 4: ARP Spoofing (respuesta ARP no solicitada) ──────────────
alert arp any any -> any any (msg:"LAB ARP Spoofing - Respuesta ARP no
solicitada"; arp.opcode:2; threshold:type both,track by_src,count 5,seconds
10; sid:9000004; rev:1;)
```

```
# Recargar las reglas:
sudo kill -USR2 $(pidof suricata)
```

(4) Hardening de Suricata + Auditoría EVE JSON

⏱ 20 min

Configurar threshold para evitar falsos positivos

Un IPS mal calibrado genera tantas alertas que el analista deja de atenderlas. El threshold limita la frecuencia de alertas por la misma firma:

Configuración de threshold

```
# Crear fichero de threshold global
sudo nano /etc/suricata/threshold.conf

# Ejemplo: suprimir alertas SYN repetidas del mismo origen (máx. 1 cada 60s)
suppress gen_id 1, sig_id 9000001, track by_src, ip 192.168.10.10

# Activar el fichero de threshold en el yaml:
threshold-file: /etc/suricata/threshold.conf
```

Analizar el log EVE JSON simulando que se envía a un SIEM

El formato EVE JSON es el estándar para integrar Suricata con un SIEM (Elastic, Splunk, Wazuh). Practicad el análisis manual del log para entender la estructura de los eventos:

Análisis del log EVE JSON — VM Suricata

```
# Contar alertas por firma (resumen de ataques detectados)
cat /var/log/suricata/eve.json | jq -r 'select(.event_type=="alert") |
.alert.signature' | sort | uniq -c | sort -rn
     847  LAB SYN Flood detectado
     312  LAB ICMP Flood detectado
      94  LAB HTTP Flood GET excesivo
      17  LAB ARP Spoofing - Respuesta ARP no solicitada

# Ver el detalle completo de un evento de SYN Flood:
cat /var/log/suricata/eve.json | jq 'select(.alert.signature=="LAB SYN Flood
detectado")' | head -1

# Exportar resumen para el informe:
cat /var/log/suricata/eve.json | jq -r
'[.timestamp,.src_ip,.alert.signature] | @csv' > resumen_alertas.csv
```

✅ Criterio de superación del punto (4)

Threshold configurado y documentado.
Log EVE JSON con mínimo 1 evento de cada categoría (SYN Flood, ICMP Flood, HTTP Flood, ARP Spoofing).
Fichero resumen_alertas.csv generado e incluido en el documento PDF.

Guía del informe en PDF

El informe acredita que has entendido lo que has ejecutado, no solo que lo has hecho. Debe ser reproducible: otra persona debe poder repetir tu experimento leyendo tu documentación.

Sección	Contenido mínimo	Capturas obligatorias
1. Instalación y configuración	Pasos seguidos, versiones instaladas, errores encontrados y soluciones aplicadas	Suricata arrancando sin errores + ping/HTTP basal funcionando
2. Ataques DoS/DDoS	Descripción técnica de cada ataque (capa OSI, recurso que agota, mecanismo) + reglas escritas y comentadas	htop ANTES y DURANTE cada ataque + salida jq de alertas EVE
3. ARP Spoofing y MITM	Mecanismo del ataque + regla Suricata + argumentario de por qué no es suficiente (mín. 150 palabras)	arp -n en la víctima con MAC falsificada + alerta EVE del ARP Spoofing
4. Hardening y auditoría	Threshold configurado + análisis de resumen_alertas.csv + tabla comparativa ataques/alertas	Salida del recuento jq por firma
5. Conclusiones	Limitaciones del laboratorio frente a entorno real + propuesta de mejora	Sin captura (texto argumentativo)

Preguntas de análisis que debes responder en el informe

1. ¿En qué momento el servidor dejó de responder durante el SYN Flood?
    a. ¿Qué conexiones acumulaba 'ss -s'?
2. ¿Por qué el HTTP Flood es más difícil de detectar para un firewall stateful que el SYN Flood?
3. ¿Qué regla generó más alertas en el log EVE?
    a. ¿Por qué crees que fue así?
4. ¿Por qué Suricata NO puede bloquear el ARP Spoofing?
    a. ¿Qué control de red sí podría hacerlo?
5. ¿Qué diferencia existe entre el modo IDS pasivo y el modo IPS inline en términos de riesgo operativo?